

CS213 2025 Tutorial Solutions

CS213/293 UG TAs

2025

Contents

Tutorial 6 [2](#)

Tutorial 6

1. Give an implementation for the scheduling problem using Heap.

Solution: Refer the slides to know about the scheduling problem.

See Algorithm 1 for the Pseudo-code.

Use of the Heap (Min Heap here) is to select the minimum remaining processing time among the processes present at any time. Before the timestamp of any new process is added, we will keep performing the jobs (available at that moment) with the minimum remaining times. As soon as a new process is added, the current process (if any) is preempted (i.e. interrupted) and the algorithm now looks for the process with the minimum remaining time. In the end, the for loop exits only when all the jobs are completed.

Given solution will have a time complexity of $\mathcal{O}(n \log n)$. Why? Think. (Hint: Number of operations (push or pop) won't exceed $4n$.)

Algorithm 1: Shortest Remaining Time Scheduling

Data: Array (sorted) *time* containing the timestamp the i^{th} job is called and a corresponding array *duration* containing the duration of the i^{th} job. Both are of size n .

```
1 Function Schedule( $n, time, duration$ ):
2   MinHeap RemTime // A Min Heap for the remaining processing times
3   for  $i \leftarrow 0$  to  $n - 1$  do
4     RemTime.push((duration[i], i)) //Sorted by the first of the pair
5     CurrTime  $\leftarrow time[i]$ 
6     if  $i = n - 1$  then
7       | EndTime  $\leftarrow \infty$  //Some large value
8     end
9     else
10      | EndTime  $\leftarrow time[i + 1]$ 
11    end
12    while not RemTime.empty() do
13      MinRemTime, CurrIndex  $\leftarrow$  RemTime.top()
14      RemTime.pop()
15      NextCurrTime  $\leftarrow$  CurrTime + MinRemTime
16      if NextCurrTime > EndTime then
17        | print("Job", CurrIndex, "starts at", CurrTime, "and ends at", EndTime)
18        | duration[CurrIndex]  $\leftarrow$  duration[CurrIndex] - (EndTime - CurrTime)
19        | RemTime.push((duration[CurrIndex], CurrIndex))
20        | break
21      end
22      print("Job", CurrIndex, "starts at", CurrTime, "and ends at", NextCurrTime)
23      print("Job", CurrIndex, "will be completed.")
24      CurrTime  $\leftarrow$  NextCurrTime
25    end
26  end
27 end
```

2. Can a Priority Queue be implemented as a red-black tree? What advantages does a heap implementation have over a red-black tree implementation?

Solution: Yes, the priority queue can be implemented using red-black trees. Priority queue has 3 operations which RB trees can also do:

- insert: Insert can be done in an RB tree using the priority as the property of the tree
- top: Return the pointer to the rightmost element of the RB tree
- deleteMax: delete the rightmost element of the RB tree

Time complexity can be made equal for all of the above operations. (top can work in $O(1)$ time. Think. Can just store the pointer to the rightmost node.)

The advantage of heap implementation is you can store the priority queue in an array, in which you aren't storing anything extra like child pointer, parent pointer etc, you are just storing the data of the node. Also, array implementation increases the cache performance.

3. Give a tree, if exists, that is a binary search tree, is a heap, and has more than two nodes. If such a tree does not exist, give a reason.

Solution: Consider the given binary tree:-



Since all keys are equal (and last level is complete hence no worry about being left-filled), this tree satisfies the definitions of both BST and Heap (Refer the slides for the definitions).

4. Suppose we have a 2D array where we maintain the following conditions: for every (i, j) , we have $A(i, j) \leq A(i + 1, j)$ and $A(i, j) \leq A(i, j + 1)$. Can this be used to implement a priority queue?

Solution: Yes, it is very similar to that of a heap. There are some modifications to the heap implementation:

- Store the negative of the priority in this data structure
- $A[i, j]$ has two children $A[i, j + 1]$ and $A[i + 1, j]$
- $A[i, j]$ has two parent $A[i, j - 1]$ and $A[i - 1, j]$
- Level of $A[i, j]$ is denoted by $i + j$
- You can again define notation of level-wise left filled.
- Deletion is done similar to heap
- Insertion is done similar to heap with a modification that the max of both parent is considered for swap.
- $A[0, 0]$ is the max-priority element.

5. Given an unsorted array, find the k th smallest element using a priority queue.

Solution: Here are the steps:-

1. Push all the elements of the array in the priority queue (Min Heap one).
2. Pop k times. Answer is the k^{th} pop.

Follow up: Can you do this in $\mathcal{O}(n)$ time (Not necessarily using Heaps)?

6. Given two heaps give an efficient algorithm to merge the heaps.

Solution: Here are the steps:-

1. Concatenate the array of the heaps (or copy all the elements from both arrays to a new one).
2. Apply BuildHeap function on this new array.

Time Complexity of this algorithm is $\mathcal{O}(m + n)$, m and n being the size of the heaps. This is because both the steps above are linear in time.

7. Give an algorithm to undo the last union operation assuming there is path compression or not.

Solution: Short answer: Maintain a *history-stack* that records every change (parent/-size/rank) for the last union. To undo the last union pop and revert records until the end of the stack. For path compression, you must additionally log every parent-pointer change caused by find.

Algorithm 2: DSU without path compression

```
1 Function Union(a, b):  
2   history.clear()  
3   history.PUSH(MARK)  
4   ra  $\leftarrow$  Find(a)  
5   rb  $\leftarrow$  Find(b)  
6   if ra = rb then  
7     | return  
8   end  
9   if size[ra] > size[rb] then  
10    | swap(ra, rb)  
11  end  
12  history.PUSH(("parent", ra, parent[ra]))  
13  history.PUSH(("size", rb, size[rb]))  
14  parent[ra]  $\leftarrow$  rb  
15  size[rb]  $\leftarrow$  size[rb] + size[ra]  
16 end  
17 Function UndoLastUnion():  
18   while not history.EMPTY() do  
19     | rec  $\leftarrow$  history.POP()  
20     | if rec.type = "parent" then  
21       | parent[rec.node]  $\leftarrow$  rec.old end  
22     | else  
23       | size[rec.node]  $\leftarrow$  rec.old end  
24     | end  
25   end  
26
```

Algorithm 3: DSU with path compression

```
1 Function FindLog( $x$ ):
2   if  $\text{parent}[x] = x$  then
3     return  $x$ 
4   end
5    $\text{old} \leftarrow \text{parent}[x]$ 
6    $\text{root} \leftarrow \text{FindLog}(\text{old})$ 
7   if  $\text{parent}[x] \neq \text{root}$  then
8      $\text{history.PUSH}(\text{"parent"}, x, \text{parent}[x])$ 
9      $\text{parent}[x] \leftarrow \text{root}$ 
10  end
11  return  $\text{parent}[x]$ 
12 end

13 Function Union( $a, b$ ):
14    $\text{history.clear}()$ 
15    $\text{ra} \leftarrow \text{FindLog}(a)$ 
16    $\text{rb} \leftarrow \text{FindLog}(b)$ 
17   if  $\text{ra} = \text{rb}$  then
18     return
19   end
20   if  $\text{size}[\text{ra}] > \text{size}[\text{rb}]$  then
21      $\text{swap}(\text{ra}, \text{rb})$ 
22   end
23    $\text{history.PUSH}(\text{"parent"}, \text{ra}, \text{parent}[\text{ra}])$ 
24    $\text{history.PUSH}(\text{"size"}, \text{rb}, \text{size}[\text{rb}])$ 
25    $\text{parent}[\text{ra}] \leftarrow \text{rb}$ 
26    $\text{size}[\text{rb}] \leftarrow \text{size}[\text{rb}] + \text{size}[\text{ra}]$ 
27 end

28 Function UndoLastUnion():
29   while  $\text{not history.EMPTY}()$  do
30      $\text{rec} \leftarrow \text{history.POP}()$ 
31     if  $\text{rec.type} = \text{"parent"}$  then
32        $\text{parent}[\text{rec.node}] \leftarrow \text{rec.old}$ 
33     end
34     else
35        $\text{size}[\text{rec.node}] \leftarrow \text{rec.old}$ 
36     end
37   end
38 end
```

8. Show that complexity bounds in theorem 9.3 and theorem 9.4 are tight.

Solution:

Theorem 9.3: Running time with weighted union, Let us suppose there are n elements in all disjoint sets. The total running time of any sequence of n unions is $\mathcal{O}(n \log(n))$

Theorem 9.4: Running time of UnionFind via forest, The running time is still $\mathcal{O}(n \log(n))$.

CONSTRUCTION (COMMON TO BOTH THEOREMS). Let n be a power of two (for simplicity). We will construct a sequence of exactly n unions whose total running time is $\Omega(n \log n)$.

1. First build a single tree T of size $\frac{n}{2}$ and height $h = \lfloor \log(n/2) \rfloor$ by repeatedly unioning pairs of equal-size trees (this is possible: start from singletons and repeatedly union pairs so sizes double each time). Under weighted union (by size or by rank) this produces a tree T whose height is $\Theta(\log(n/2)) = \Theta(\log n)$.
2. Let u be some leaf of T at depth h . There remain $\frac{n}{2}$ singleton sets (the elements not yet in T). Now perform, one by one, $\frac{n}{2}$ unions each of the form

$$\text{Union}(u, s_i),$$

where s_i is a different singleton for $i = 1, \dots, \frac{n}{2}$.

COST ANALYSIS. For each union $\text{Union}(u, s_i)$ the implementation must compute $\text{Find}(u)$ and $\text{Find}(s_i)$. The cost of $\text{Find}(s_i)$ is constant (it is a singleton), while the cost of $\text{Find}(u)$ is proportional to the depth of u . Because we do *not* use path compression, the depth of u remains $\Theta(h) = \Theta(\log n)$ throughout these $\frac{n}{2}$ unions: attaching singletons as children of the (much larger) tree does not increase the depth of the existing nodes in T , and weighted union rules ensure the singleton is attached under the larger component. Thus each of the $\frac{n}{2}$ unions costs $\Theta(\log n)$ time, yielding total time

$$\frac{n}{2} \cdot \Theta(\log n) = \Theta(n \log n).$$

The initial phase of building T costs at most $O(n)$, so the whole sequence of n unions costs $\Omega(n \log n)$. Combining this lower bound with the theorems' upper bounds $O(n \log n)$ shows the bounds are tight:

$$\text{total time} = \Theta(n \log n).$$

9. Give an implementation of $\text{printSet}(x)$ function in UnionFind (with path compression) that prints the set containing x . You may add one field in each node and must not alter the asymptotic running times of the other operations.

Solution:

IDEA. Add a single extra pointer field *next* next to each node. Interpret the *next* pointers as a circular linked list that contains exactly the members of the node's current disjoint set. Keep the invariant that every set's circular list contains its current representative (root). Initially each singleton's *next* points to itself. When two sets are unioned we concatenate their circular lists in $\mathcal{O}(1)$ time by swapping two *next* pointers. We do *not* change *next* during path compression, so path compression and union-by-rank/size retain their original amortized complexity.

Algorithm 4: Union-Find with next circular-list for constant-time concatenation and path compression

Data: Integer n (number of elements). We maintain arrays $\text{parent}[0..n-1]$, $\text{rank}[0..n-1]$ and $\text{next}[0..n-1]$.

```
1 Function MakeSet( $n$ ):
2   for  $v \leftarrow 0$  to  $n - 1$  do
3      $\text{parent}[v] \leftarrow v$   $\text{rank}[v] \leftarrow 0$ 
4      $\text{next}[v] \leftarrow v$ 
5   end
6 end
7 Function Find( $x$ ):
8   if  $\text{parent}[x] \neq x$  then
9      $\text{parent}[x] \leftarrow \text{Find}(\text{parent}[x])$ 
10  end
11  return  $\text{parent}[x]$ 
12 end
13 Function Union( $x, y$ ):
14    $r_x \leftarrow \text{Find}(x)$ 
15    $r_y \leftarrow \text{Find}(y)$ 
16   if  $r_x = r_y$  then
17     return
18   end
19   if  $\text{rank}[r_x] < \text{rank}[r_y]$  then
20      $\text{swap}(r_x, r_y)$ 
21   end
22    $\text{parent}[r_y] \leftarrow r_x$ 
23   if  $\text{rank}[r_x] = \text{rank}[r_y]$  then
24      $\text{rank}[r_x] \leftarrow \text{rank}[r_x] + 1$ 
25   end
26    $t \leftarrow \text{next}[r_x]$ 
27    $\text{next}[r_x] \leftarrow \text{next}[r_y]$ 
28    $\text{next}[r_y] \leftarrow t$ 
29 end
30 Function printSet( $x$ ):
31    $r \leftarrow \text{Find}(x)$   $y \leftarrow r$ 
32   repeat
33      $\text{print}(y)$ 
34      $y \leftarrow \text{next}[y]$ 
35   until  $y = r$ ;
36 end
```
